

DRIFTS: Robustness for Time Series is Feasible, Distributed, and Fun

Abstract—As complex machine learning models are increasingly integrated into critical systems, ensuring their robustness, transparency, and trustworthiness becomes essential. Explainability methods that offer formal and verifiable guarantees about model behavior are crucial not only for interpretability but also for robust decision assurance. In this work, we address the problem of generating formally grounded and robustness-aware explanations for classifiers operating in the time series domain. Our main objective is to introduce the concept of a continuous anti reason. We formalize this concept as an Interval Constraint Function (ICF) which, given a classifier and a sample, assigns an interval for each feature value of the sample. An ICF that constitutes a continuous anti reason for a given sample ensures that all data points whose feature values lie within the corresponding intervals are classified differently from the sample class. Each continuous anti reason is associated with a cost, quantifying the extent or perturbation required to induce a class change. The higher the cost of a continuous anti reason, the more robust the classifier is on that sample. Therefore such a property is paramount for precisely characterizing the robustness boundaries of the classifier on a sample-wise basis. Given that computing a continuous anti reason is CONP-complete, we develop the framework called *Robustness for Time Series is Feasible, Distributed, and Fun (DRIFTS)*, a distributed algorithm that leverages an in-memory key-value store to coordinate multiple workers. The effectiveness and scalability of our approach are validated empirically on Random Forest models using univariate time series datasets from the UCR Archive [1].

I. INTRODUCTION

Machine learning has become a cornerstone of modern data analysis, powering applications in many diverse domains such as cyber-security [2], [3], industrial automation [4], healthcare [5], [6], and more [7]. Nevertheless, despite the rapid proliferation of adaptations and applications in recent years, the opaque nature of many machine learning methods remains a fundamental challenge. Consequently, explainability has garnered increasing attention, although a universally accepted definition has yet to be established [8]. As models are increasingly employed to support decision-making in strategic domains such as healthcare [9], security [10], and finance [11], it has become impossible to overlook their lacking in transparency when it comes to evaluate their decisions or classifications. Accordingly, the growing role of artificial intelligence has heightened the need for good explanations [12] and robustness in classifier decisions [13]. Motivated by this need, the present work aims to deliver a framework for *robustness* for time series classification. Indeed, time series data present a unique challenge for robustness, as they consist of a large number of features that individually convey limited information but gain interpretive and meaningful value in the context of their temporal dependencies.

Within the machine learning paradigms, Random Forests (RF) [14] represent a widely adopted methodology for data mining applications [15]. Their ability to handle heterogeneous data types and large feature spaces makes them appealing for complex problems, including time series classification. Despite their widespread success, they suffer from a well-known limitation: their structure acts as a “black box” [16], making it difficult to understand the reasoning behind individual classifications. This lack of interpretability limits the adoption of RF and in critical domains, where explainability, robustness, and trust are essential requirements [17].

The primary goal of this paper is to address this gap by developing a new framework called *Robustness for Time Series is Feasible, Distributed, and Fun (DRIFTS)* to compute classifier robustness by continuous anti reasons for RF classifiers. A continuous anti reason is defined as a set of feature value constraints that guarantees the classification different from the sample label. Identifying these reasons provides formal and verifiable robustness of a classifier decision, enabling end-users to understand not just what the model classified but how many sample perturbation are required to change the classification. However, the underlying problem remains CO-NP-complete. This is unsurprising, as explainability problems, even in the context of decision trees, are known to be NP-hard [18].

The remainder of this paper is structured as follows. Section II surveys the relevant literature on reason computation and robustness methods for decision trees and RF. Section III gives a formal representation and definitions of our setting for solving the continuous anti reason problem establishing their mathematical properties. Section IV presents the framework and the experimental results, analyzing their outcomes and implications. Section V concludes the paper with a summary of key findings and insights.

II. RELATED WORK

This section overviews research related to our work, focusing on the formal robustness of tree ensembles and on recent advancements in explainability and robustness for time series classification highlighting both similarities with existing approaches and the novel contributions of our method.

Our objective is to provide formal, verifiable robustness guarantees at the sample level. We take advantage of the *sufficient reason* concepts [19], where the authors introduced the notions of sufficient reasons—the minimal set of instance characteristics that justify a classifier decision—and complete reasons, encompassing all characteristics that can trigger that decision. These are computed by transforming

classifiers into Ordered Binary Decision Diagrams (OBDDs) [20], and later into Decision-DNNFs [21], enabling linear-time computation of complete reasons for Boolean classifiers, then generalized by [22]. Choi et al. [23] demonstrated that the input-output behavior of models such as neural networks and random forests can be represented symbolically via Boolean encodings, enabling formal reasoning and *sample wise robustness* [21]. However, unlike Darwiche sufficient reason and Choi encoding—defined for discrete tabular data and prone to discontinuities—our approach ensures continuous, gap-free regions suited to time series analysis. Wicker et al. [24] provide method to certify the robustness against adversarial attacks on model inputs or parameters. By propagating symbolic intervals through the network, the method calculates a formal upper bound on the largest possible change to an explanation. In contrast, we provide a post-hoc formal analysis working directly on features intervals. Verifying robustness of tree ensembles is generally NP-hard [25]. In [26] the authors analyze complexity for probabilistic δ -sufficient-reasons for decision tree models proving that finding minimal δ -sufficient-reasons is NP-hard. However, our methodology is deterministic and the class complexity is CO-NP-complete. However, in [27] it is proposed a new training paradigm called “verifiable learning” to construct ensembles that are provably verifiable in polynomial time. Our approach differs, as we provide a post-hoc formal analysis for trained models. Other work seeks to scale verification, such as [28], which introduces a heuristic to verify robustness for multi-class tree ensembles. Our methodology is deterministic and applied to the time series domain. Formal methods have also been applied to certify robustness against different threats, as in [29], which uses abstract interpretation to certify robustness to data bias. Our approach instead provides a precise, sample-specific robustness guarantee rather than a model-wide certificate or heuristic. Our concept of an continuous anti reason is related to, but distinct from, counterfactual explanations. For example, RobX [30] generate robust counterfactuals by iteratively refining it by the new introduced metric. However, a counterfactual is typically a single point that flips the classification. Our continuous anti reason is a more powerful concept: it defines the entire continuous region of perturbations that guarantee a class change, thus providing a complete view of the decision boundary near the sample. Finally, other works like [31] also link robustness and explainability for time series, but do so by creating simplifications of the input and combining balance among error, simplicity and robustness. This simplification let the authors find solutions in polynomial time. Our method, by contrast, operates directly on the original feature space offering a more precise and formally grounded explanation of the model behavior without needing simplifications but introducing apriori based early stopping mechanisms.

III. CONTINUOUS ANTI REASON: A GENERAL DEFINITION

In this section, we discuss the concepts that form the theoretical foundation of our work. In particular, we introduce

the notions of the Interval Constraint Function (ICF) and the Endpoint Universe (EU). These concepts are essential for defining the notions Continuous Anti Reason (CAR). Then, we will address the concept of ICF cost. Finally, we formalize the concept of robustness and the problem addressed in this paper.

We start introducing the fundamental concepts characterizing our solution space, which allow us to determine the continuous anti reason associated with a given sample, considering both the classifier and its structural properties. Let *Features* be a finite set of features. The set of all open intervals ($\mathbb{I}(\mathbb{R})$) is defined as $\mathbb{I}(\mathbb{R}) = \{(r_1, r_2) : r_1 < r_2 \subseteq \{-\infty, +\infty\} \cup \mathbb{R}\}$. We can now introduce the concept of Interval Constraint Function:

Definition 1 (Interval Constraint Function): The Interval Constraint Function is a function $ICF : Features \rightarrow \mathbb{I}(\mathbb{R})$.

Informally, the ICF maps each feature to a corresponding open interval. For each feature, the ICF specifies an interval of admissible values, excluding the interval endpoints. Since the two endpoints differ by definition, no ambiguity arises. Notice that $(-\infty, +\infty)$ is an accepted interval that contains all the possible feature values. For example, consider the IRIS dataset [32] with *Features* = {sepal length, sepal width, petal length, petal width} the ICF function for these Features map them to the following intervals: sepal length $\rightarrow (4.5, 5.0)$, sepal width $\rightarrow (3.1, 3.6)$, petal length $\rightarrow (1.1, 1.6)$, petal width $\rightarrow (0.1, 0.8)$.

To make the dimensionality of our search space finite, we introduce the concept of the Endpoint Universe (EU).

Definition 2 (Endpoint Universe): The Endpoint Universe is a function $EU : Features \rightarrow 2^{\mathbb{R}}$ where for each $f \in Features$ the set $EU(f)$ is finite.

Intuitively, EU represents the set of all possible endpoints (excluding $-\infty$ and $+\infty$) that can be used to form intervals for each feature. Although the total number of possible intervals for a feature f is quadratic in $|EU(f)|$, it remains finite since EU is finite. For example, consider again the feature petal length from the IRIS dataset. The set $EU(\text{petal length})$ may include endpoints such as $\{1.0, 1.1, 1.2, \dots, 1.7\}$. Despite being potentially large, this set is still finite. The following definition restricts the space of ICFs to those built on a given EU.

Definition 3 (EU respected by ICF): An ICF respects EU if and only if for all $f \in Features$, letting $ICF(f) = (b, e)$, we have $b, e \in EU(f) \cup \{-\infty, +\infty\}$.

It is easy to see that the space of ICFs that respect EU is finite. From now on, we fix EU and assume that every ICF we introduce respects it. Notice that we must include the infinities ($-\infty$ and $+\infty$) in Definition 3 for ICF, since they are not considered in Definition 2.

As ICFs are a central structure in this work, we now introduce and explain the key operations defined on them. The first operation is the *intersection* between two ICFs.

Definition 4 (ICF Intersection): Given two interval constraint functions ICF and ICF' , we say that $\overline{ICF}$ is the intersection of ICF and ICF' , denoted $\overline{ICF} = ICF \cap ICF'$, if and only if $\forall f \in Features, \overline{ICF}(f) = ICF(f) \cap ICF'(f)$,

where the intersection of two intervals (b, e) and (b', e') is $(\max(b, b'), \min(e, e'))$ if $\max(b, b') \leq \min(e, e')$, and $\emptyset$ otherwise. If any feature yields $\emptyset$, then $\overline{\text{ICF}}$ does not exist. Consider two ICFs defined over features from the IRIS dataset:

$$\text{ICF}(\text{petal length}) = (3, 8), \quad \text{ICF}'(\text{petal length}) = (5, 10)$$

$$\text{ICF}(\text{sepal width}) = (2, 4), \quad \text{ICF}'(\text{sepal width}) = (1, 3)$$

The intersection $\text{ICF} \cap \text{ICF}'$ exists since all features have overlapping intervals:

$$\text{ICF}(\text{petal length}) \cap \text{ICF}'(\text{petal length}) = (5, 8)$$

$$\text{ICF}(\text{sepal width}) \cap \text{ICF}'(\text{sepal width}) = (2, 3)$$

Now consider ICF with a different ICF'' :

$$\text{ICF}(\text{petal length}) = (3, 8), \quad \text{ICF}''(\text{petal length}) = (5, 10)$$

$$\text{ICF}(\text{sepal width}) = (2, 4), \quad \text{ICF}''(\text{sepal width}) = (6, 9)$$

Computing the intersection feature-wise:

$$\text{ICF}(\text{petal length}) \cap \text{ICF}''(\text{petal length}) = (5, 8)$$

$$\text{ICF}(\text{sepal width}) \cap \text{ICF}''(\text{sepal width}) = \emptyset$$

Since sepal width yields $\emptyset$ (the intervals $(2, 4)$ and $(6, 9)$ do not overlap), the intersection $\text{ICF} \cap \text{ICF}''$ does not exist.

Another important operation is the *feature extension*.

Definition 5 (Feature Extension): Given interval constraint functions ICF and $\overline{\text{ICF}}$, and a feature $f \in \text{Features}$, we say that $\overline{\text{ICF}}$ is a feature extension of ICF (either right $\overrightarrow{f}$ or left $\overleftarrow{f}$) if and only if the following conditions hold:

- $\forall f' \in \text{Features}$, if $f' \neq f$, then $\text{ICF}(f') = \overline{\text{ICF}}(f')$
- Let $\text{ICF}(f) = (b, e)$ and $\overline{\text{ICF}}(f) = (\bar{b}, \bar{e})$. Then:
 - right extension ($\overrightarrow{f}$): $\bar{b} = b$ and $\bar{e} = \min\{r \in \text{EU}(f) \cup \{+\infty\} : r > e\}$;
 - left extension ($\overleftarrow{f}$): $\bar{b} = \max\{r \in \text{EU}(f) \cup \{-\infty\} : r < b\}$ and $\bar{e} = e$.

Informally speaking, we are now expanding the ICF with feature f . To extend the ICF along f , we must first ensure that for all features $f' \neq f$, the intervals remain unchanged (i.e., $\text{ICF}(f') = \overline{\text{ICF}}(f')$). Then, we have the two possible cases: expanding to the right (i.e., we are raising the high end) or to the left (i.e., we are lowering the starting point of the interval). In the first case, we keep fixed the start b and extend e to the next point in the EU (of f) or to $+\infty$. Same goes for the right extension. An example of left and right expansion is visible in Figure 1. Notice that an ICF cannot be right-extended ($\overrightarrow{f}$) if $\text{ICF}(f) = (b, +\infty)$ for some $b \in \mathbb{R} \cup -\infty$. Similarly, it cannot be left-extended ($\overleftarrow{f}$) if $\text{ICF}(f) = (-\infty, e)$ for some $e \in \mathbb{R} \cup +\infty$.

For two intervals (b, e) and (b', e') , we have $(b, e) \subseteq (b', e')$ if and only if $b' \leq b \leq e \leq e'$. Finally, we define the domination relation among ICFs.

Definition 6 (Domination): Given two ICFs, ICF and ICF' , over the same domain Features , we say that ICF' dominates ICF , denoted $\text{ICF} \leq \text{ICF}'$, if and only if $\forall f \in \text{Features}$, $\text{ICF}(f) \subseteq \text{ICF}'(f)$.

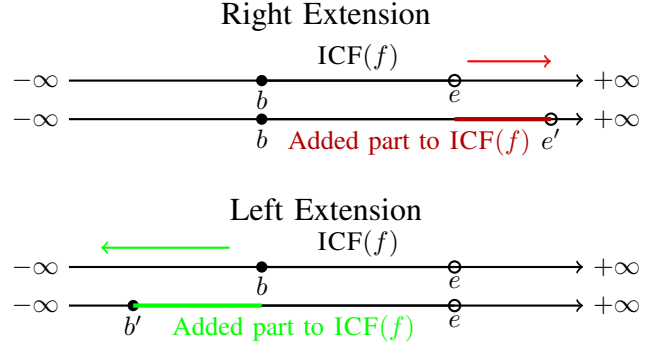


Fig. 1: Comparison of the original interval value $\text{ICF}(f)$ (black) and its $\overleftarrow{f}$ -extension (top, red) and $\overrightarrow{f}$ -extension (bottom, green). The newly added intervals are highlighted in color.

$$\text{ICF}(f_1) = (1, 2), \text{ICF}'(f_1) = (1, 1.5), \text{ICF}''(f_1) = (1.25, 3)$$

The containment relationships for f_1 are:

$$(1, 2) \supseteq (1, 1.5)$$

but $(1, 2)$ and $(1.25, 3)$ are incomparable: $(1, 2) \not\supseteq (1.25, 3)$ and $(1.25, 3) \not\supseteq (1, 2)$. Here, ICF dominates ICF' but ICF and ICF'' are incomparable under domination.

A sample x is a function $x : \text{Features} \rightarrow \mathbb{R}$. Let $\mathbb{X}$ be the set of all possible samples, where each sample $x \in \mathbb{X}$ is a function $x : \text{Features} \rightarrow \mathbb{R}$ that assigns a real value to each feature. Let $Y = \{y_1, y_2, \dots, y_k\}$ be a finite set of labels. A classifier C is a function $C : \mathbb{X} \rightarrow Y$ that maps each sample to a label.

We now introduce the concept of continuous anti reason that is fundamental for this work, as it forms the basis of the problem addressed.

Definition 7 (Continuous Anti Reason (AR)): Given a classifier C such that and a class label y , an ICF is an *continuous anti reason* for y in C if and only if $\forall x' \in \text{ICF}, C(x') \neq y$.

Informally, the intuition behind a AR is as follows: for all features whose values lie within the corresponding intervals defined by the ICF, the classifier C produces an output y' that differs from the classification $y = C(x)$ of the sample x . It should be noted that an ICF may correspond to multiple classes, as its purpose is not to classify the samples within its intervals as y . In other words, the classification remains stable for a class different from y under any perturbation of the samples constrained within the intervals of the ICF. Moreover, it is important to emphasize that while x may partially lie within the intervals of the ICF (as visible in Figure 2), the two must eventually diverge—otherwise, if x were entirely contained within the ICF, such an ICF could instead represent a continuous reason.

The *continuous reason* (CR), conversely, defines the corresponding intervals of an ICF within which the classification remains stable for the same class as x under any perturbation

of x constrained to those intervals. Given a classifier C and a sample x such that $C(x) = y$, the set of all possible ARs is denoted as $AR_{C,y}$.

It is also important to clarify what we mean by a *continuous non-reason*.

Definition 8 (Continuous Non-Reason (NR)): Given a sample x , its classification $y = C(x)$ made by a classifier C and an ICF such that $x \in \text{ICF}$, we say that ICF is a *continuous non-reason* for x in C if and only if $\exists x' \in \text{ICF } \text{Label}(C, x') \neq y$.

Intuitively, a NR identifies an ICF where it is possible to “draw” at least one sample (i.e., one possible assignment of features within the ICF intervals) that receives a classification different from y . The concepts of CR, AR, and NR are illustrated in Figure 2.

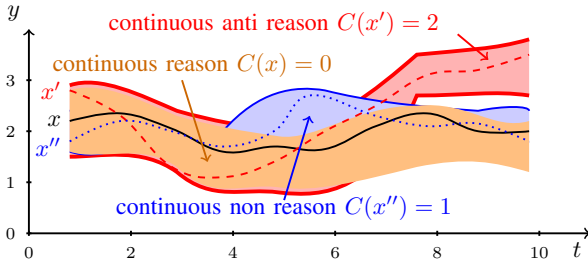


Fig. 2: An example of a **continuous reason** (orange), a **continuous anti reason** (red), and a **continuous non reason** (blue). The sample x with label 0 corresponds to the one used to construct the continuous reason.

In the following, we first introduce the general formulation of the cost function associated with an ICF. Subsequently, we specify the particular cost function adopted in our framework.

Let ICF be the set representing the set of all possible ICFs fixed features. We now define the cost function.

Definition 9 (Cost Function): Given a sample x and an ICF, a *cost function* $\text{cost}_x : \text{ICF} \rightarrow \mathbb{R}^+$, giving to each $\text{ICF} \in \text{ICF}$ a cost c . To be a valid cost function, it must satisfy the following property: it must be monotonic non-decreasing within the interval increasing dimension.

Definition 9 is intentionally broad, as the general nature of the proposed framework makes it undesirable to restrict its potential applications. The cost function cost_x can, in fact, incorporate information specific to the context in which the classifier operates. The cost for one ICF is $\text{cost}_x(\text{ICF})$.

We now formally define how we calculate $\text{cost}_x(\text{ICF})$. Given a training set $T = \{t_1, \dots, t_n\} \subset \mathbb{X}$, a sample x we define the difference vector D as

$$D = \begin{cases} \delta_i^+ = t_i[f] - x_i[f] & \text{if } x_i[f] > t_i[f] \\ \delta_i^- = x_i[f] - t_i[f] & \text{otherwise} \end{cases} \quad \forall t_i \in T, \forall f \in \text{ICF}$$

We identify all the positive values for a feature f in D as δ_f^+ and all the negative ones as δ_f^- . Moreover, thanks to this calculus we obtain a particularly useful property as the mean on D μ_D is equal to 0.

Let $\sigma_{x,f}^+ = \sqrt{\frac{\sum_{i \in \delta_f^+} (\delta_{f_i}^+)^2}{\delta_f^+}}$ and $\sigma_{x,f}^- = \sqrt{\frac{\sum_{i \in \delta_f^-} (\delta_{f_i}^-)^2}{\delta_f^-}}$ be the positive and negative standard deviations of a sample feature f , respectively. Let $p_{x,f}^+ = \frac{|\delta_f^+|}{|D_f|}$ and $p_{x,f}^- = \frac{|\delta_f^-|}{|D_f|}$ denote the positive and negative ratios in D . Then, the area under the curve for feature f in ICF is defined as

$$A_{x,f}(b, e) = p_{x,f}^+ \int_b^e \mathcal{N}(x; 0, (\sigma_{x,f}^+)^2) dx + p_{x,f}^- \int_b^e \mathcal{N}(x; 0, (\sigma_{x,f}^-)^2) dx$$

where

$$\mathcal{N}(x; 0, (\sigma_{x,f}^*)^2) = \frac{1}{\sqrt{2\pi(\sigma_{x,f}^*)^2}} e^{-x^2/(2(\sigma_{x,f}^*)^2)}$$

Given an ICF, the cost function $\text{cost}_x(\text{ICF})$ for an ICF given a sample x is obtained by summing the areas for each feature given the interval centered in the sample feature value that is:

$$\text{cost}_x(\text{ICF}) = \sum_{f \in \text{Features}} A_{x,f}(b - x[f], e - x[f]) \text{ s.t. } \text{ICF}(f) = (b, e)$$

This cost function was selected because it is independent of the classifier, it is always defined within the range $[0, 1]$ for each feature f (i.e. $A_{x,f}(0, 0) = 0 \leq A_{x,f}(b, e) \leq A_{x,f}(-\infty, +\infty) = 1$) and $A_{x,f}(b, e)$ represents proportionally what the classifier has seen for that feature, and, in the context of robustness analysis, it assigns higher penalties to deviations occurring in regions that are more frequently observed within the training dataset.

Example Cost Function Consider the sample x with classification $C(x) = 0$, and the AR ICF depicted in Figure 2. We now describe the computation of $\text{cost}_x(\text{ICF})$. First, for a given training dataset $T \subset \mathbb{X}$, we analyze the distribution of each feature $f \in \text{Features}$ relative to the sample x . We compute difference vector D_f . From this resulting array, the positive and negative standard deviations, $\sigma_{x,f}^+$ and $\sigma_{x,f}^-$, are calculated together with the positive and negative ratios $p_{x,f}^+$ and $p_{x,f}^-$, as illustrated in Figure 3a. This procedure yields a Gaussian-type curve for each feature, centered at 0 (representing the value of $x[f]$). Then, we take the corresponding interval from the ICF (i.e., $\text{ICF}(f) = (b, e)$) and normalize it relative to x by subtracting $x[f]$, yielding the interval $(b - x[f], e - x[f])$. The cost component for this feature $A_{x,f}$ is defined as the area under the curve. This process is repeated for all features $f \in \text{Features}$, as shown in Figure 3b (where this process features is represented for the features represented as black, blue, and green points). As visualized in the figure, if the ICF contains the sample x , the normalized interval contains 0. Conversely, if the ICF does not contain x for a given feature (e.g., the green feature), the interval will lie entirely above or below 0. Finally, the total $\text{cost}_x(\text{ICF})$ is computed by summing the individual cost components from all features: $\text{cost}_x(\text{ICF}) = \sum_{f \in \text{Features}} A_{x,f}(\text{ICF})$.

Then, we move to the robustness concept. As also defined in [21] the robustness is defined sample wise. We define robustness as:

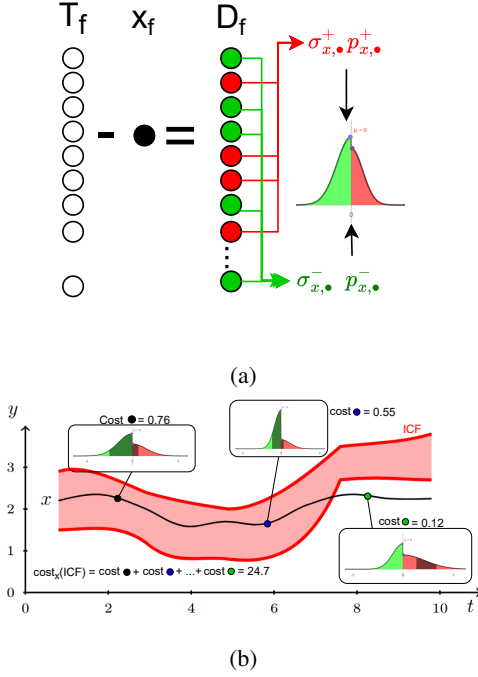


Fig. 3: Overview of the cost computation process given an ICF and a sample x . In 3a the illustration of the cost components for feature f . The **Difference Vector** D_f (left) is created by subtracting the **Sample** $x[f]$ (middle) from the feature f values of the **Training Set** $T[f]$ (right). Red points represent positive values and green represent negative ones. The area proportions (p^- , p^+) and standard deviations ($\sigma_{x,f}^-$, $\sigma_{x,f}^+$) are then calculated from this resulting vector. In ?? an example of how the cost function $cost_x(ICF)$ is computed on an ICF and a sample x .

Definition 10 (Robustness): Given a classifier C a sample x its classification $y = C(x)$ the *robustness* of x in C is:

$$r(C, x) = 1 - \frac{\max_{ICFs \in AR_{C,y}} cost_x(ICF)}{|Features|}$$

Intuitively, the robustness measure quantifies how resistant a sample is to classification changes. Specifically, the AR represents the smallest perturbation required to alter the classification to $C(x) \neq y$. The measure lies within the interval $[0, 1]$: a value of 0 indicates no robustness (i.e., the classifier decision changes easily the sample classification), while a value of 1 denotes maximum robustness (i.e., the sample is completely stable within its neighborhood). This behavior arises from the value of $\max_{ICFs \in AR_{C,y}} cost_x(ICF)$, which determines the extent of permissible perturbation before a classification change occurs.

Finally, we provide the general problem tackled in this work.

Problem 1: Given a classifier $C : \mathbb{X} \rightarrow Y$, an endpoint universe EU and a sample $x \in \mathbb{X}$ determine whether a ICF is a continuous anti reason for $C(x)$.

It can be demonstrated that an AR is generally not unique, even when x , C , and EU are fixed. That is the main reason behind the introduction of the cost function and in particular the property that as the interval increases the cost increases. The complexity of identifying such reasons depends on the classifier. For the experimental (Section IV) perspective we consider the Random Forest classifier, that exhibit CO-NP-complete complexity in solving Problem 1.

Theorem 1 (CO-NP-completeness): Problem 1 for random forests is CO-NP-complete.

The proof of Theorem 1 can be establish by via a a polynomial-time reduction from the NP-hard *Max2Sat* problem [33]. However, for the sake of brevity—and because it lies beyond the scope of this paper—the detailed proof is omitted.

IV. DRIFTS: OVERVIEW AND EXPERIMENTAL RESULTS

We now present an overview of the *DRIFTS* framework architecture, with a particular focus on the distributed algorithm that constitutes its core. This discussion will be followed by the experimental evaluation.

Framework *DRIFTS* framework relies on the following fundamental observation about the monotonicity properties of AR in random forests.

Lemma 1 (Monotonicity of Anti-reasons): Let RF be a random forest and y be a label, then: (i) if ICF is a AR for y in RF , then every $ICF' \subseteq ICF$ (i.e., $ICF'(f) \subseteq ICF(f)$ for all $f \in Features$) is also an AR for y ; (ii) conversely, if ICF' is not an AR for y in RF , then any $ICF \supseteq ICF'$ is also not an AR for y .

This monotonicity property enables an efficient search strategy. Our algorithm adopts a *randomized distributed Apriori-like approach*: workers independently explore the search space until the stopping condition is met: the ICFs contained in an AR already verified by another process.

We now describe the operational framework of the proposed algorithms. In our setting, a key-value store is employed to create a shared accessible context and to coordinate multiple workers during the parallel search for continuous anti reasons.

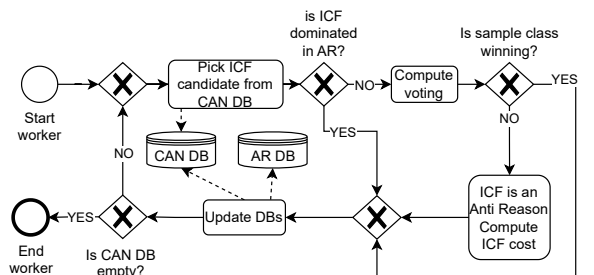


Fig. 4: Diagram on how the *DRIFTS* framework works.

Figure 4 present how the *DRIFTS* framework works. Once initialization is complete, the workers operate in parallel to explore candidate reasons. Each worker enters a loop, retrieving a candidate c from the **CAN** key-value database. To prevent redundant work, it first checks if c has already been processed and stored in the **AR** (its correctness derives from the previous Lemma 1) If this check fails, the worker performs the expensive check operation. Based on the result, c is added to **AR** and the cost of the . This structure, using the key-value store as a *shared remote dictionary*, enables all workers to efficiently query, update, and extend the shared knowledge base, avoiding concurrency conflicts and repeated calculations during the distributed search. The main process waits for all workers to finish (i.e., when the **CAN** database is empty) and then gathers the final results from the **AR** database.

Experimental Results To evaluate the effectiveness of the proposed methodology we conducted a series of experiments on selected datasets from the UCR Archive [1], the standard benchmark for time series classification. A subset of datasets was chosen based on their structural and temporal complexity. The experiments were organized by progressively varying the classifier complexity, namely the number of trees in the Random Forest (RF), and the length of the time series instances. This preliminary evaluation focused on the datasets in Table I. For each dataset, a Random Forest classifier was trained using Bayesian optimization, and the best-performing configuration was selected. To efficiently manage the large volume of intermediate and output data, we employed the in-memory, multithreaded key-value database *KeyDB*¹, a high-performance fork of Redis², designed to overcome Redis single-threaded limitations. All experiments were performed on a system equipped with an Intel Core i9-10980HK CPU (2.40 GHz), 32 GB RAM, running Ubuntu 22.04.3 LTS, and utilizing 32 parallel workers. While the current experiments were executed on a single system, the algorithm is fully compatible with a distributed workers configuration.

Table I presents experimental results demonstrating the effectiveness of our distributed algorithm. The **Filtered Rate** metric shows that pruning via ICFsubsumption and profile dominance eliminates 6.7%–94.1% of unnecessary verification checks across datasets. The algorithm handles billions of iterations (up to 223M for Melbourne) while maintaining practical runtimes through the memorization strategies. The **Filtered Rate** varies significantly across datasets, ranging from 6.7% (MiddlePhalanxOutlineCorrect) to 94.1% (SonyAIBORobotSurface1), correlating with dataset complexity and the structure of the endpoint universe. Datasets with larger endpoint universes (higher **Mean EU Features**) generally benefit more from profile-based pruning, as incomparable ICFs are more common in high-dimensional spaces. The algorithm performs billions of continuous anti reason check iterations (e.g., 196M for Wine, 223M for

Melbourne) while maintaining tractable runtime through aggressive pruning, validating the practical utility of our theoretical framework.

Code reproducibility All source code, experimental scripts, and configuration files used in this study are provided as supplementary material. Detailed instructions for setting up the environment, processing the data, and running the experiments and the analysis are provided in the README.md file. The UCR Time Series Archive datasets are available at <https://www.timeseriesclassification.com/dataset.php>.

V. CONCLUSIONS

This paper addresses the growing challenge of trust in complex “black-box” machine learning classifiers like Random Forests by introducing the concept of continuous anti reason, which is used to provide a robustness measure about a model behavior within a specific region of the feature space. Then, we developed *DRIFTS* a distributed algorithm explicitly designed to leverage core data management techniques leveraging an in-memory key-value store for scalability while avoiding redundant computations. This contributions offer several benefits. They provide a mathematically rigorous framework for reasoning about classifier decisions robustness enabling practical deployment in distributed environments. Future extensions will explore adaptations to other ensemble architectures such as deep learning methods and validation extensions.

ACKNOWLEDGMENT

We would like to thank the donors and curators of the data to the UCR Time Series Archive <https://www.timeseriesclassification.com/dataset.php>.

REFERENCES

- [1] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, “The ucr time series archive,” 2019.
- [2] I. J. Vourganas and A. L. Michala, “Applications of machine learning in cyber security: a review,” *Journal of Cybersecurity and Privacy*, vol. 4, no. 4, pp. 972–992, 2024.
- [3] K. Shaikat, S. Luo, V. Varadharajan, I. A. Hameed, and M. Xu, “A survey on machine learning techniques for cyber security in the last decade,” *IEEE access*, vol. 8, pp. 222310–222354, 2020.
- [4] T. A. Khan, S. M. Ali, K. M. Ali, A. Aziz, S. Ahmad, A. Anwar, and S. A. Khan, “Harnessing artificial intelligence for optimum performance in industrial automation,” *Engineering Proceedings*, vol. 76, no. 1, p. 105, 2025.
- [5] Q. An, S. Rahman, J. Zhou, and J. J. Kang, “A comprehensive review on machine learning in healthcare industry: classification, restrictions, opportunities and challenges,” *Sensors*, vol. 23, no. 9, p. 4178, 2023.
- [6] W. K. Wang, I. Chen, L. Hershkovich, J. Yang, A. Shetty, G. Singh, Y. Jiang, A. Kotla, J. Z. Shang, R. Yerrabelli, *et al.*, “A systematic review of time series classification techniques used in biomedical applications,” *Sensors*, vol. 22, no. 20, p. 8016, 2022.
- [7] I. H. Sarker, “Machine learning: Algorithms, real-world applications and research directions,” *SN computer science*, vol. 2, no. 3, p. 160, 2021.
- [8] R. Marcinkevičs and J. E. Vogt, “Interpretability and explainability: A machine learning zoo mini-tour,” *arXiv preprint arXiv:2012.01805*, 2020.
- [9] R. Macri and S. L. Roberts, “The use of artificial intelligence in clinical care: A values-based guide for shared decision making,” *Current Oncology*, vol. 30, no. 2, pp. 2178–2186, 2023.

¹Documentation available at <https://docs.keydb.dev/>

²Documentation available at <https://redis.io/>

Dataset	1	2	3	4	5	6	7	8	9	10	11
Train Size	5.7e1	1.0e3	6.0e2	1.14e3	4.67e2	3.60e3	2.0e1	5.0e2	1.0e3	2.0e2	1.0e2
Test Size	5.4e1	6.16e3	2.91e2	2.32e3	3.84e3	1.32e3	6.01e2	4.50e3	3.70e2	2.05e3	1.0e2
Series Length	2.34e2	1.52e2	8.0e1	2.4e1	1.66e2	5.0e2	7.0e1	1.4e2	2.71e3	1.31e2	9.6e1
N Estimators	1.0e1	1.0e1	1.0e1	1.0e1	1.0e1	1.0e1	1.7e1	5.4e1	5.9e1	9.2e1	1.01e2
Mean EU Features	3.45	4.67	1.80e1	6.07e1	1.08e1	1.76e1	3.31	5.66	3.25	9.38	4.04
EU Std	7.10e-1	1.63	5.73	1.05e1	3.72	4.12	5.83e-1	2.14	5.58e-1	2.66	1.15
Total Time (ms)	5.80e5	3.55e7	1.47e6	2.75e7	1.71e7	1.42e7	9.31e4	2.19e7	6.00e6	3.44e7	4.59e7
ICF Checks	2.16e3	6.17e3	9.76e2	7.79e3	8.24e3	2.49e3	3.27e4	6.83e3	5.14e3	5.49e3	3.51e3
Anti Reason Check Iteration	2.36e5	1.96e8	2.28e5	2.23e8	5.82e7	1.81e7	3.73e4	1.18e8	2.46e6	1.17e8	1.38e7
IterGoodRatio (%)	20.5	100.0	89.9	100.0	100.0	100.0	71.6	100.0	76.3	100.0	87.0
IterBadRatio (%)	79.5	0.0	10.1	0.0	0.0	0.0	28.4	0.0	23.7	0.0	13.0
Early Stop	1.17e5	9.66e7	1.10e5	1.04e8	2.85e7	8.92e6	7.31e3	5.80e7	1.07e6	5.72e7	6.75e6
Early Stop from Good	2.43e4	9.66e7	9.86e4	1.04e8	2.85e7	8.92e6	5.99e3	5.80e7	8.04e5	5.72e7	5.88e6
Early Stop from Bad	9.23e4	0	1.12e4	0	0	0	1.32e3	0	2.68e5	0	8.75e5
Filtered Rate (%)	41.6	14.2	6.7	10.3	24.8	14.5	94.1	20.5	91.1	9.7	28.4

TABLE I: Experimental results of the distributed algorithm for computing continuous anti reasons across the 11 time series datasets. The table reports dataset characteristics (train/test size, series length, number of estimators in the random forest), endpoint universe statistics (mean and standard deviation of features in EU_{RF}), computational metrics (total time, number of ICF checks, anti reason check iterations), early stopping effectiveness (good/bad profile hits), and the overall filtered rate showing the percentage of ICF checks avoided through pruning (for the column we have 1 = Wine, 2 = Wafer, 3 = MiddlePhalanxOutlineCorrect, 4 = MelbournePedestrian, 5 = ChlorineConcentration, 6 = FordA, 7 = SonyAIBORobotSurface1, 8 = ECG5000, 9 = HandOutlines, 10 = FacesUCR, 11 = ECG200).

- [10] A.-A. Bouramdane, "Cyberattacks in smart grids: challenges and solving the multi-criteria decision-making for cybersecurity options, including ones that incorporate artificial intelligence, using an analytical hierarchy process," *Journal of Cybersecurity and Privacy*, vol. 3, no. 4, pp. 662–705, 2023.
- [11] A. Khunger, "Deep learning for financial stress testing: A data-driven approach to risk management," *International Journal of Innovation Studies*, 2022.
- [12] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," 2018.
- [13] A. Tocchetti, L. Corti, A. Balayn, M. Yurrita, P. Lippmann, M. Brambilla, and J. Yang, "Ai robustness: a human-centered perspective on technological challenges and opportunities," *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–38, 2025.
- [14] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.
- [15] V. Y. Kulkarni and P. K. Sinha, "Random forest classifiers: a survey and future research directions," *Int. J. Adv. Comput.*, vol. 36, no. 1, pp. 1144–1153, 2013.
- [16] M. Affenzeller, B. Burlacu, V. Dorfer, S. Dorl, G. Halmerbauer, T. Königswieser, M. Kommenda, J. Vetter, and S. Winkler, "White box vs. black box modeling: On the performance of deep learning, random

- forests, and symbolic regression in solving regression problems,” in *International Conference on Computer Aided Systems Theory*, pp. 288–295, Springer, 2019.
- [17] B. Li, P. Qi, B. Liu, S. Di, J. Liu, J. Pei, J. Yi, and B. Zhou, “Trustworthy ai: From principles to practices,” *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–46, 2023.
 - [18] G. Audemard, S. Bellart, L. Bounia, F. Koriche, J.-M. Lagniez, and P. Marquis, “On the explanatory power of decision trees,” 2021.
 - [19] A. Darwiche and A. Hirth, “On the reasons behind decisions,” in *ECAI 2020*, pp. 712–720, IOS Press, 2020.
 - [20] R. E. Bryant, “Graph-based algorithms for boolean function manipulation,” *Computers, IEEE Transactions on*, vol. 100, no. 8, pp. 677–691, 1986.
 - [21] A. Darwiche and A. Hirth, “On the (complete) reasons behind decisions,” *Journal of Logic, Language and Information*, vol. 32, no. 1, 2022.
 - [22] C. Ji and A. Darwiche, “A new class of explanations for classifiers with non-binary features,” 2023.
 - [23] A. Choi, A. Shih, A. Goyanka, and A. Darwiche, “On symbolically encoding the behavior of random forests,” *CoRR*, vol. abs/2007.01493, 2020.
 - [24] M. Wicker, J. Heo, L. Costabello, and A. Weller, “Robust explanation constraints for neural networks,” *arXiv preprint arXiv:2212.08507*, 2022.
 - [25] A. Kantchelian, J. D. Tygar, and A. Joseph, “Evasion and hardening of tree ensemble classifiers,” in *International conference on machine learning*, pp. 2387–2396, PMLR, 2016.
 - [26] M. Arenas, P. Barceló, M. Romero Orth, and B. Subercaseaux, “On computing probabilistic explanations for decision trees,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 28695–28707, 2022.
 - [27] S. Calzavara, L. Cazzaro, G. E. Pibiri, and N. Prezza, “Verifiable learning for robust tree ensembles,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1850–1864, 2023.
 - [28] L. Devos, L. Cascioli, and J. Davis, “Robustness verification of multi-class tree ensembles,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, pp. 21019–21028, 2024.
 - [29] A. Meyer, A. Albarghouthi, and L. D’Antoni, “Certifying robustness to programmable data bias in decision trees,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 26276–26288, 2021.
 - [30] S. Dutta, J. Long, S. Mishra, C. Tilli, and D. Magazzeni, “Robust counterfactual explanations for tree-based ensembles,” in *International conference on machine learning*, pp. 5742–5756, PMLR, 2022.
 - [31] J. A. Telle, B. A. T. Håvardstun, and C. Ferri, “Optimal robust simplifications for explaining time series classifications,” 2024.
 - [32] R. A. Fisher, “Iris,” UCI Machine Learning Repository, 1936. DOI: <https://doi.org/10.24432/C56C76>.
 - [33] M. R. Garey, D. S. Johnson, and L. Stockmeyer, “Some simplified np-complete problems,” in *Proceedings of the sixth annual ACM symposium on Theory of computing*, pp. 47–63, 1974.